Cloudflare Tunnel + Docker Compose sur macOS

Guide synthétique — application React, Python et C++ composée d'environ dix conteneurs, avec API REST et MQTT. Point d'entrée local supposé : http://localhost:8080.

Prérequis

Docker Compose fonctionne sur le Mac ; l'application est accessible sur http://localhost:8080. Installer cloudflared via Homebrew :

```
brew install cloudflared
cloudflared --version
```

Option A — Tunnel lancé séparément (recommandé pour tester)

Dans le dossier du projet, démarrer les services ; puis lancer le tunnel dans un second Terminal :

```
docker compose up -d
# Dans un autre Terminal :
cloudflared tunnel --url http://localhost:8080
```

Copier l'adresse HTTPS *.trycloudflare.com affichée dans les logs et la tester depuis un téléphone en réseau mobile. Pour arrêter : Ctrl+C dans le terminal du tunnel. L'adresse change généralement au prochain lancement.

Option B — Tunnel intégré à Docker Compose

Ajouter le service ci-dessous au fichier Compose existant, en remplaçant web par le nom réel du service Docker qui écoute sur le port 8080. Le service cloudflared doit partager son réseau Docker.

```
services:
  cloudflared:
    image: cloudflare/cloudflared:latest
    command: tunnel --no-autoupdate --url http://web:8080
    restart: unless-stopped
    depends_on:
      - web
```

Démarrer avec docker compose up -d, puis retrouver l'URL publique avec :

```
docker compose logs -f cloudflared
```

Attention : dans un conteneur, localhost désigne ce conteneur, pas le Mac ni un autre service Docker. Utiliser le nom de service Docker comme web:8080.

REST, MQTT et sécurité

Les échanges REST et MQTT entre conteneurs peuvent rester privés sur le réseau Docker. En revanche, les appels effectués par React dans le navigateur ne doivent pas viser localhost ou des noms Docker internes : préférer des chemins relatifs comme /api, routés par un reverse proxy. Si React se connecte directement à MQTT, prévoir un accès WebSocket sécurisé (WSS) adapté.

Un Quick Tunnel est public sans authentification intégrée : ne pas exposer de données sensibles, d'outils d'administration ou de broker MQTT directement. Pour des tests privés, préférer un tunnel nommé avec Cloudflare Access ou une authentification applicative.

Dépannage et choix

502 : vérifier le port, le service cible et les logs Docker. Interface visible mais API cassée : vérifier les URL dans l'onglet Réseau du navigateur. MQTT cassé : distinguer communications internes et MQTT depuis le

navigateur.

Choix conseillé : option A pour des tests ponctuels ; option B pour démarrer l’ensemble en une commande. Pour une URL stable et un contrôle d’accès, passer ensuite à un tunnel nommé.

Documentation officielle : https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/ et https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/do-more-with-tunnels/trycloudflare/